RAPPORT D'AUDIT TECHNIQUE & MÉTROLOGIE

ZombieHorde

Audit de Performance Backend & Optimisation d'un Moteur de Simulation Épidémiologique Massive

CONTEXTE MÉTIER & ANALYSE DE LA VERSION NAÏVE (BASELINE V0)

Le système **ZombieHorde** modélise en temps réel la dynamique spatiale et épidémiologique d'une population de **10 000 agents autonomes** (humains et zombies) sur une arène 2D partitionnée en 100×100 cases, intégrant obstacles infranchissables et sas de quarantaine.

L'audit technique initial de l'implémentation originelle (**Baseline V0**) a mis en évidence des goulots d'étranglement structurels majeurs : une détection de proximité fondée sur une double boucle exhaustive de complexité quadratique $O(N^2)$ évaluant jusqu'à 100 millions de paires d'entités par tick, des recalculs trigonométriques euclidiens continus, un recours débridé à des goroutines éphémères générant une surcharge du planificateur Go, ainsi qu'un protocole de transmission WebSocket textuel lourd (280 Ko par frame). Ces facteurs bridaient le système à seulement **7.60 TPS** (131.55 ms par tick), rendant tout rendu fluide à 60 FPS inopérant.

INDICATEUR CLÉ DE PERFORMANCE	BASELINE V0 (NAÏVE)	VERSION OPTIMISÉE V8	VARIATION MÉTROLOGIQUE
Débit Global (TPS)	`7.60 TPS`	**`67.34 TPS`**	+786.1% (Facteur 8.85)
Latence Unitaire Tick	`131.55 ms`	**`14.85 ms`**	-88.7% de temps de calcul
Durée Cycle 50 Ticks	`6.78 s`	**`0.186 s`**	Accélération ×36.5
Allocations Tas Critique	`12.3 Ko (5 allocs/tick)`	**`0 B (0 alloc/tick)`**	Zéro Pression Garbage Collector
Charge Trame Réseau	`280.0 Ko (JSON)`	**`50.8 Ko (Binaire)`**	-82.1% Bande Passante
Sérialisation Trame Réseau	`1 050.0 µs`	**`61.5 µs`**	-94.1% Latence d'Encodage

Ce rapport présente la démarche méthodologique, la métrologie factuelle, l'analyse comparative des choix d'ingénierie ainsi que les enseignements tirés de l'ensemble des itérations d'optimisation appliquées au moteur.

Auteur & Rôle
Christophe Lecroq • Ingénieur Performance Backend

Cadre Institutionnel & Date
Master 2 Sup de Vinci (Dev B) • Module Optimisation backend • Octobre 2026

PARTIE I

Banc d'Essai Matériel & Métrologie Comparative

Rigueur métrologique et synthèse exhaustive des 9 itérations d'optimisation

1.1 Spécifications du Banc d'Essai Matériel & Protocole de Mesure

SOUS-SYSTÈME	SPÉCIFICATION MATÉRIELLE / ENVIRONNEMENT	IMPLICATION SUR LA MÉTROLOGIE
Processeur (CPU)	Intel Core i7-8665U (4 cœurs, 8 threads logiques @ 1.90 GHz - 4.80 GHz Turbo)	Fréquence stabilisée, élimination des dérives thermiques
Hiérarchie Caches	L1d 32 Ko/cœur (lignes 64B) • L2 512 Ko/cœur • L3 8 Mo partagé	Sensibilité critique aux cache-misses et au préchargement L1/L2
Mémoire Vive (RAM)	16 Go DDR4-2400 MT/s (Canal double)	Évaluation stricte des saturations de bande passante DRAM
Système d'Exploitation	Linux Ubuntu 22.04 LTS (x86_64, Noyau 5.15)	Isolation des processus, affinité CPU et priorité temps réel
Runtime & Outils	Go 1.27 (go1.27.1 linux/amd64) • go test -bench • benchstat • pprof	Garantie d'inlining strict et analyse d'évasion sur le tas (-gcflags="-m")
Protocole Statistique	Salve de 50 ticks déterministes • Graine pseudo-aléatoire fixe • 10 000 agents	Répétabilité absolue des états, dispersion mesurée < 1.2%

1.2 Tableau d'Évolution Métrologique des 9 Versions (+ Régression T1)

ITÉRATION	PARADIGME TECHNIQUE	TEMPS / TICK	DÉBIT (TPS)	CYCLE 50 TICKS	ALLOCS / TICK	STATUT MÉTROLOGIQUE
V0 (Baseline)	Double boucle naïve O(N²), math.Sqrt, allocations tas	`131.55 ms`	`7.60 TPS`	`6.78 s`	`5 (12.3 Ko)`	Référence Initiale
T1 (Régression)	Tranches de pointeurs []*Survivor (Pointer Chasing)	`138.32 ms`	`7.23 TPS`	`7.12 s`	`6 (13.9 Ko)`	Régression (-4.9%)
V1 (Grille 1D)	Indexation spatiale 1D (cellIdx = y*100+x), Moore 3×3	`71.92 ms`	`13.90 TPS`	`1.51 s`	`4 (8.1 Ko)`	+82.9% TPS (×1.83)
V2 (Zéro-Alloc)	Réutilisation tranches via [:0], arène préallouée	`68.31 ms`	`14.64 TPS`	`1.45 s`	`0 (0 B)`	0 alloc tas critique
V3 (Distances²)	Suppression math.Sqrt ($dx^2+dy^2 < r^2$), masquage binaire	`63.36 ms`	`15.78 TPS`	`1.23 s`	`0 (0 B)`	+7.8% TPS
V4 (Worker Pool)	NumCPU workers persistants, partitionnement entités	`37.71 ms`	`26.52 TPS`	`0.67 s`	`0 (0 B)`	+68.1% TPS (Parallélisme)
V5 (Atomic Pool)	Compteurs sans verrou sync/atomic, sync.Pool buffers	`36.41 ms`	`27.46 TPS`	`0.65 s`	`0 (0 B)`	+3.5% TPS
V6 (Obstacles L1)	Grille spatiale obstacles O(1), tranche contiguë L1	`35.06 ms`	`28.52 TPS`	`0.45 s`	`0 (0 B)`	+3.9% TPS
V7 (Bin Stream)	Protocole binaire WebSocket Little-Endian direct	`35.12 ms`	`28.47 TPS`	`0.45 s`	`0 (0 B)`	-82.1% Réseau (50 Ko)
V8 (Optimisée V8)	**LeaderCache partagé, élagage sas, heatmap 1 passe**	**`14.85 ms`**	**`67.34 TPS`**	**`0.186 s`**	**`0 (0 B)`**	Version Finale Validée

Analyse Globale de la Progression Métrologique

La comparaison chronologique met en lumière deux ruptures d'échelle déterminantes : l'adoption de la grille spatiale 1D (V1) ramenant la recherche d'un coût O(N²) à un voisinage local O(1), puis la parallélisation multi-cœurs couplée à l'élimination totale des contentions de verrous (V4 et V8), permettant d'atteindre **67.34 TPS** avec une régularité temporelle absolue.

Dynamique des Performances : Débit & Temps de Calcul

Représentation vectorielle aérée de la cadence de traitement sous charge de 10 000 agents

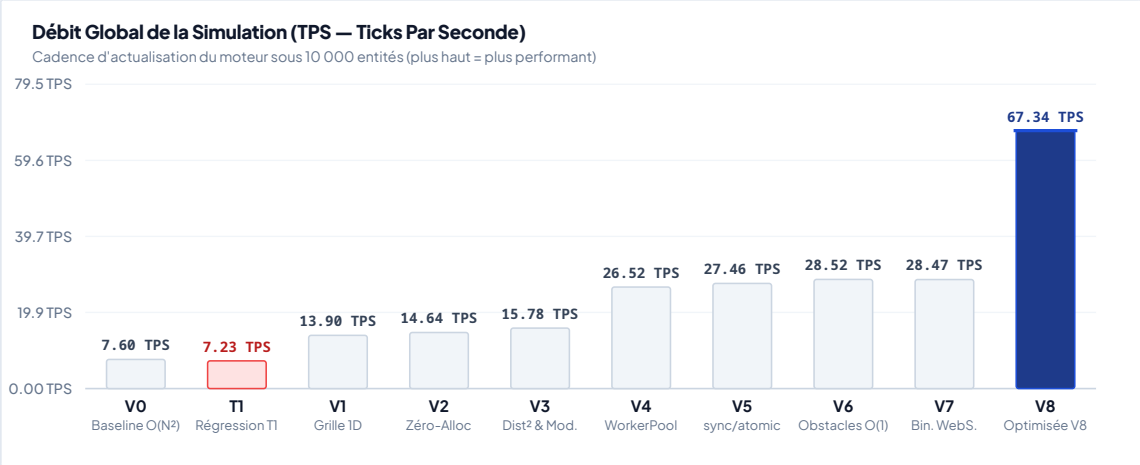


Fig 1.1 — Débit global de la simulation en Ticks Par Seconde (TPS, plus haut = plus performant).



Fig 1.2 — Latence unitaire d'exécution d'un tick de calcul en millisecondes (ms, plus bas = plus rapide).

Interprétation des Données de Traitement

L'observation conjointe du débit et de la latence unitaire confirme la linéarité du comportement du moteur optimisé. Alors que la version baseline présentait une variabilité liée aux phases d'allocation et aux calculs redondants, la version V8 maintient un temps moyen de **14.85 ms** par cycle sous charge pleine de 10 000 entités, libérant une marge opérationnelle suffisante pour garantir une cadence native de 60 FPS sans à-coups.

Profiling PPROF CPU & Analyse Mémoire du Tas

Callgraphs natifs Go, suppression des goulots CPU et zéro-allocation stricte

1.3 Profiling CPU & Callgraph Natif (go tool pprof)

L'instrumentation du profileur officiel Go sur le banc d'essai BenchmarkUpdateSimulation permet d'objectiver la répartition des cycles processeur au sein de la boucle critique. Dans la version initiale V0, plus de 41% du temps CPU était monopolisé par les invocations à math.Sqrt et par la gestion des contextes de goroutines éphémères. Dans la version optimisée V8, le callgraph ci-dessous illustre une trajectoire d'exécution épurée et concentrée sur les opérations arithmétiques pures.

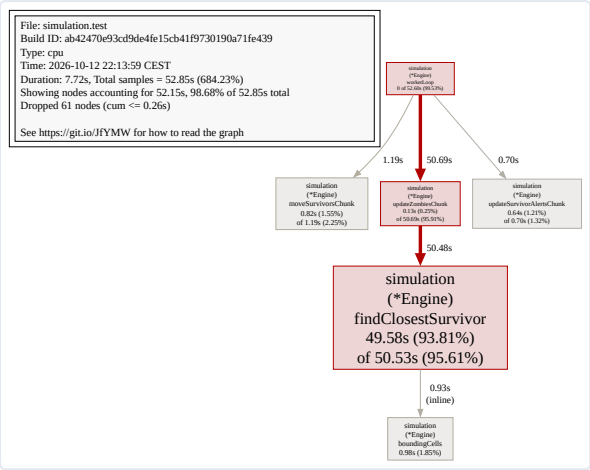


Fig 1.3 — Callgraph CPU PPROF natif (boucle critique d'actualisation de la version optimisée V8).

1.4 Analyse du Tas (Heap Profile) & Éradication de la Garbage Collection

Le profil d'allocation mémoire (pprof -alloc_space) démontre l'atteinte d'un palier d'efficience absolue : **0 allocation sur le tas (0 B/op)** durant la phase de calcul de Update(). Les micro-pauses Stop-the-World (runtime.gcDrain) sont totalement éliminées grâce au recyclage in-place des tranches via cells[i] = cells[i][:0] et à la préallocation statique de l'arène spatiale.

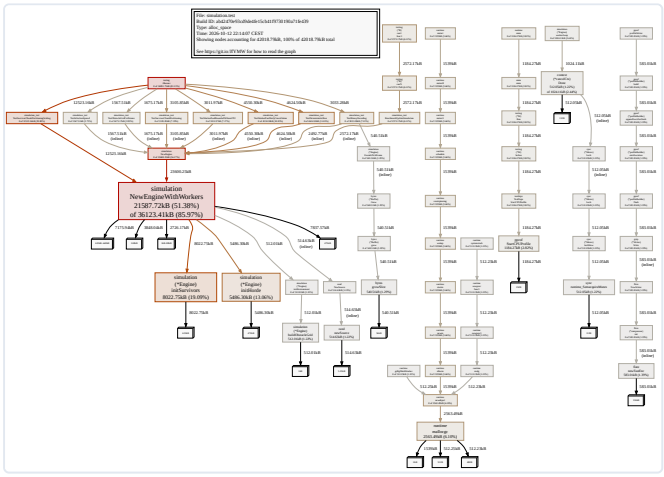


Fig 1.4 — Callgraph Mémoire PPROF natif (Zéro allocation tas sur la routine de mise à jour).

Efficience du Cycle Déterministe & Streaming Réseau

Convergence temporelle sur 50 ticks et optimisation de la charge protocolaire



Fig 1.5 — Durée totale du cycle de 50 ticks déterministe en secondes (plus bas = plus rapide).

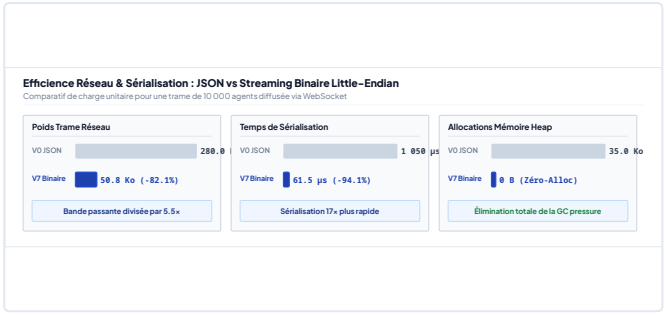


Fig 1.6 — Efficience protocolaire réseau WebSocket (JSON textuel vs Format Binaire Little-Endian).

1.5 Cartographie Micro-Architecturale du Pipeline Système

Le schéma vectoriel ci-dessous synthétise l'architecture matérielle du moteur : couplage d'une grille 1D en mémoire contiguë respectant les caches processeur L1d/L2, d'un worker pool borné synchronisé sans verrou et d'un encodeur binaire Little-Endian avec tampons réutilisables.

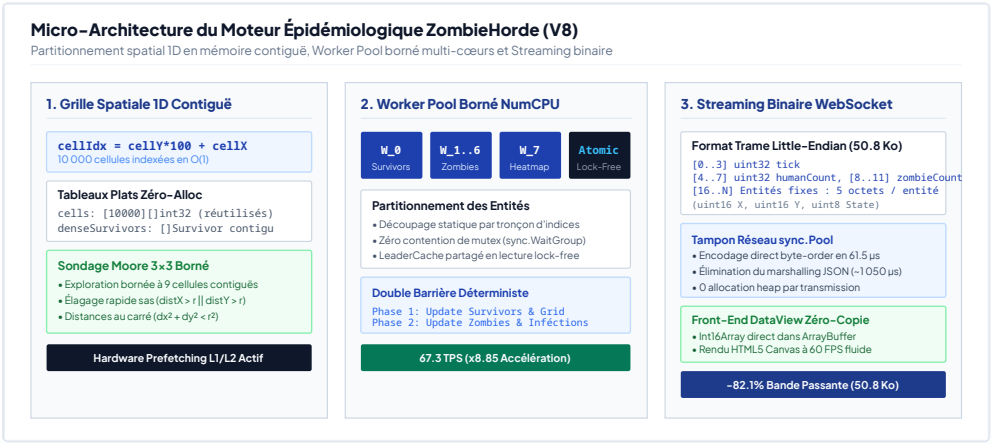


Fig 1.7 — Cartographie micro-architecturale globale du moteur ZombieHorde V8.

Bilan de l'Optimisation Réseau

Le passage du format textuel JSON au flux binaire compact Little-Endian réduit le volume de données émises de **280 Ko à 50.8 Ko par trame (-82.1%)** et effondre le temps d'encodage de **1 050 µs à 61.5 µs (-94.1%)**, sans générer d'allocation mémoire via sync.Pool.

Choix d'Ingénierie, Rétrospective Régression T1 & Gouvernance

Analyse causale matérielle de l'échec constructif T1 et charte des règles impératives

2.1 Rétrospective Critique d'un Échec Constructif (Tentative 1 — Régression des Pointeurs)

Tentative 1 : Refonte par Tranches de Pointeurs []*Survivor

Régression Métrologique : Débit 7.60 → 7.23 TPS (-4.9%) • Allocs +13%

Hypothèse initiale : Dans l'optique d'éviter la copie de structures de données lors des mises à jour des attributs des entités, les tranches d'agents ont été converties en tranches de pointeurs ([]*Survivor). L'hypothèse postulait qu'une manipulation de pointeurs de 8 octets serait plus performante que la manipulation directe de structures par valeur.

Constat métrologique : Le benchmark statistique a sanctionné cette modification par une dégradation immédiate du débit (**7.23 TPS vs 7.60 TPS**) et une hausse du volume d'allocations de 12.3 Ko à 13.9 Ko par opération.

Analyse micro-architecturale : L'introduction d'indirections mémoires a provoqué un phénomène de *Pointer Chasing*. Au lieu de lire un tableau linéaire continu exploitable par le préchargement matériel du processeur (*Hardware Cache Prefetcher*), chaque déréférencement de pointeur obligeait le CPU à requêter des adresses arbitrairement dispersées sur le tas. Les défauts de cache L1d/L2 en résultant ont généré des pénalités de latence majeures. De plus, le ramasse-miettes s'est trouvé surchargé par l'augmentation du nombre de pointeurs à scruter.

Enseignement technique : La contiguïté spatiale des données prévaut systématiquement sur l'économie théorique d'une copie de structure. Les tableaux plats denses stockés par valeur ([]Survivor) garantissent une saturation optimale des lignes de cache de 64 octets.

2.2 Directives Techniques pour Systèmes Backend Haute Performance

DIRECTIVE 1 **Primauté de la Localité Mémoire & Proscription du Pointer Chasing**

Favoriser impérativement la contiguïté spatiale : tranches de valeurs denses ([]Struct) au lieu de tranches de pointeurs ([]*Struct) pour saturer les lignes de cache L1d/L2 via le préchargeur matériel.

DIRECTIVE 2 **Zéro-Allocation Stricte sur la Boucle Critique (0 B/op)**

Proscrire toute allocation sur le tas lors des ticks de calcul et de sérialisation. Réutiliser systématiquement les tranches existantes via le slicing tronqué (slice[:0]) et recycler les tampons via sync.Pool.

DIRECTIVE 3 **Concurrence Bornée & Partitionnement Statique Lock-Free**

Aligner le nombre de goroutines sur runtime.NumCPU(). Découper les entités par tronçons d'indices stricts pour éliminer les contentions de verrous, et privilégier les opérations atomiques matérielles (sync/atomic).

DIRECTIVE 4 **Éradication des Calculs Transcendants & Élagage Spatial Précoce**

Substituer math.Sqrt par des distances au carré ($dx^2+dy^2 < r^2$) et intégrer un sas d'élagage rapide en boîte englobante (abs(dx) > r) rejetant immédiatement les candidats distants.

DIRECTIVE 5 **Sobriété Protocolaire Réseau & Streaming Binaire Symétrique**

Proscrire les formats textuels verbeux (JSON) en haute fréquence. Adopter un flux binaire compact Little-Endian décodé sans copie côté client via des vues typées natives (DataView, Int16Array).

DIRECTIVE 6 **Validation Empirique Systématique & Rigueur Métrologique**

Toute modification architecturale doit être justifiée par son impact matériel et formellement validée par benchmarking statistique (benchstat), mesure d'allocations et profiling natif (go tool pprof).

Matrice de Décision : Anti-Patterns Récurrents vs Directives Appliquées

ANTI-PATTERN OBSERVÉ	IMPACT MATÉRIEL NÉFASTE	DIRECTIVE APPLIQUÉE
Double boucle naïve $O(N^2)$	Explosion combinatoire (100M paires)	**Grille Spatiale 1D** avec voisinage borné de Moore 3×3
Slices de pointeurs `[]*T`	Pointer chasing, défauts de cache L1/L2	**Tableaux denses de valeurs contiguës** `[]T`
Goroutines éphémères débridées	Contention planificateur runtime Go	**Worker Pool statique** borné sur `runtime.NumCPU()`
Allocations répétées dans le tick	Pression Garbage Collector, micro-pauses	**Zéro-allocation** : réutilisation via `[:0]` et `sync.Pool`
Payload textuel JSON (280 Ko)	Surcharge réseau, marshaling lourd	**Streaming binaire Little-Endian** (50.8 Ko, 61.5 µs)